DAA Assignment 3 (CO3): Divide and Conquer

CSA0617 · Design and Analysis of Algorithms

---

1. Introduction and Experimental Setup

This report answers the seven questions of Assignment 3 (CO3) by designing each divide-and-conquer (D&C) algorithm, deriving its cost, implementing it in Python, and checking the derived cost against instrumented measurements. Every number in the tables below was produced by the scripts listed in the Appendix, run with the fixed seed 2401.

Table 1.1: Team details

Role	Name	Registration number
Student A	Jerry David R	192424401
Student B	Jason Joshva M J	192511209

Table 1.2: Experimental configuration

Parameter	Value
Language	Python 3.12.3 (CPython)
Random seed	2401 (random.Random(2401) in every script)
Machine	1 vCPU Intel Xeon @ 2.10 GHz, 4 GB RAM, Linux 6.18
Timing method	time.perf_counter(), median of repeated runs (reps stated per question)
Primary metric	Operation counts (comparisons, multiplications, distance calculations), which do not depend on the machine
Secondary metric	Wall-clock time in ms, indicative only
Source files	common.py, q1.py to q7.py

Operation counts are the main evidence because they match the recurrences directly. Wall-clock times in Python include interpreter overhead, so they are used only to locate crossover regions and are read together with the counts. Where an observed value differs from its prediction, the report explains the difference instead of adjusting the prediction.

2. Q1: Decomposition and Elimination in Ordered Data

2.1 Part A: Finding the Maximum and Minimum

Shared group work. Finding the maximum and the minimum independently costs 2(n − 1) comparisons: one pass for each. Divide and conquer saves comparisons by letting one comparison serve both answers.

- **Divide:** split a[lo..hi] at the middle into a left and a right half.
- **Base cases:** one element returns (a, a) with 0 comparisons; two elements need exactly 1 comparison to order the pair, which gives both its minimum and its maximum.
- **Each subproblem returns:** the pair (minimum, maximum) of its range.
- **Combine:** compare the two minima and the two maxima: 2 comparisons.

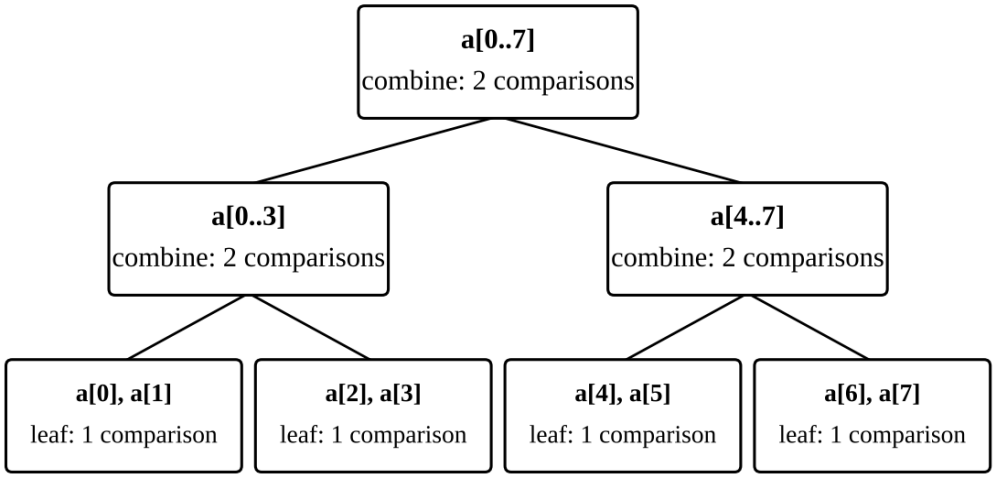
```
def maxmin_dc(a, lo, hi, cnt):
    if lo == hi:        # 1 element: 0 comparisons
        return a[lo], a[lo]
    if hi == lo + 1:        # 2 elements: 1 comparison
        cnt[0] += 1
        return (a[lo], a[hi]) if a[lo] < a[hi] else (a[hi], a[lo])
    mid = (lo + hi) // 2
    mn1, mx1 = maxmin_dc(a, lo, mid, cnt)
    mn2, mx2 = maxmin_dc(a, mid + 1, hi, cnt)
    cnt[0] += 2        # combine: min vs min, max vs max
    return min(mn1, mn2), max(mx1, mx2)
```

Student A: design and mathematical analysis.

The recursion tree for n = 8 shows how subproblems are formed and where comparisons are spent. Each leaf is a pair; each internal node adds 2 comparisons.

Student B: experimental validation.

Datasets: random integers (seed 2401), ascending, descending, and all-equal arrays, for n from 8 to 1,000,000. Every comparison of two array elements is counted by a counter passed through the recursion. Results are checked against Python's min and max.



Total for n = 8: 4 × 1 + 2 × 2 + 1 × 2 = 10 comparisons

Figure 2.1: Recursion tree for n = 8 (comparisons per node)

For n = 8 this is 4 leaf comparisons + 2 × 2 + 1 × 2 = 4 + 4 + 2 = 10 comparisons, against 2(8 − 1) = 14 for two separate passes.

The recurrence follows from the three steps (two half-size subproblems, then 2 comparisons to combine):

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \quad (n > 2), \\ T(2) &= 1, \quad T(1) = 0 \end{aligned}$$

For $n = 2^k$, unrolling the recurrence $k - 1$ times gives

$$\begin{aligned} T(n) &= 2^{k-1}T(2) + 2\left(2^{k-2} + \cdots + 2 + 1\right) \\ &= 2^{k-1} + 2\left(2^{k-1} - 1\right) = 3n/2 - 2 \end{aligned}$$

Compared with independent search, which needs $2(n - 1) = 2n - 2$:

$$\begin{aligned} (2n - 2) - (3n/2 - 2) &= n/2 \text{ comparisons saved,} \\ \frac{3n/2 - 2}{2n - 2} &\to 3/4 \end{aligned}$$

So divide and conquer needs about 75% of the comparisons of the independent method, a saving of 25% for large n. Both methods are $\Theta(n)$; the improvement is in the constant. The saving exists because a single comparison of a pair decides both its minimum and its maximum.

Table 2.1: Comparisons for Max/Min (random data; other datasets gave identical counts)

n	Naive 2(n−1)	Predicted ⌈3n/2⌉−2	D&C, split at middle	D&C, split at power of two
8	14	10	10	10
64	126	94	94	94
100	198	148	162	148
1,000	1,998	1,498	1,510	1,498
1,024	2,046	1,534	1,534	1,534
10,000	19,998	14,998	15,902	14,998
100,000	199,998	149,998	165,534	149,998
1,000,000	1,999,998	1,499,998	1,524,286	1,499,998

Findings and the discrepancy.

1. For powers of two (8, 64, 1,024) the observed count equals $3n/2 - 2$ exactly.
2. The count does not depend on the data: random, ascending, descending and all-equal inputs gave the same number, because the algorithm's comparisons are fixed by the recursion shape.
3. For other n the middle split exceeds the prediction (n = 100: 162 against 148; n = 10^6: 1,524,286 against 1,499,998, which is 1.6% more). The prediction $3n/2 - 2$ assumes every leaf is a pair. A split at the middle produces some single-element leaves, which cost 0 comparisons but still force a 2-comparison combine above them.
4. In general the count is (number of pair leaves) + 2 × (number of leaves − 1). If all leaves are pairs (n even) this is $3n/2 - 2$; if one leaf is a singleton (n odd) it is $\lceil 3n/2 \rceil - 2$. Splitting so that the left part is the largest power of two below n gives exactly these leaf counts, and the last column confirms the formula $\lceil 3n/2 \rceil - 2$ holds for every n tested.

2.2 Part B: Binary Search

Shared group work. The input is a sorted array. Compare the target with the middle element: on equality stop, on a smaller target continue in the left half, otherwise in the right half. The base case is an empty range (low > high), which means the target is absent.

```
def bsearch(a, lo, hi, t, cnt):
    if lo > hi: return -1        # empty range: not found
    mid = lo + (hi - lo) // 2
    cnt[0] += 1        # one three-way comparison with a[mid]
    if a[mid] == t: return mid
    if t < a[mid]: return bsearch(a, lo, mid - 1, t, cnt)
    return bsearch(a, mid + 1, hi, t, cnt)
```

Counting convention: one probe of a[mid] with a three-way outcome (equal, smaller, larger) counts as 1 comparison.

Student B: elimination logic.

- **Decision rule:** compare t with a[mid]. If t = a[mid], return. If t < a[mid], keep [low, mid − 1]. If t > a[mid], keep [mid + 1, high].
- **What is eliminated:** after an unsuccessful probe, the probed element and every element on the far side of it: the whole right part [mid, high] when t < a[mid], or the whole left part [low, mid] when t > a[mid]. That is at least half of the remaining range.
- **Why the target cannot be there:** the array is sorted, so every element at index ≥ mid is ≥ a[mid] > t. None of them can equal t. The argument for the left part is symmetric. This is the loop invariant: if t is in the array, it lies in a[low..high].
- **Termination:** the range size strictly decreases at every call (to at most $\lfloor n/2 \rfloor$). The recursion ends when the target is found or when low > high, which happens after at most $\lfloor \log_2 n \rfloor + 1$ probes.
- **Efficiency:** each probe discards half of the candidates, so $T(n) = T(n/2) + 1$ with $T(1) = 1$, which solves to $T(n) = \lfloor \log_2 n \rfloor + 1$ in the worst case. Doubling n adds only one probe, while linear search would double its work.

Student A: experimental investigation.

The array is a[i] = 2i, so every even number is present and every odd number (including values below the minimum and above the maximum) is guaranteed absent. Successful cases search for existing elements (all elements when n ≤ 20,000, otherwise 20,000 random elements); unsuccessful cases search for random odd values.

Table 2.2: Binary search probes

n	Bound $\lfloor \log_2 n \rfloor + 1$	Max (found)	Avg (found)	Max (absent)	Avg (absent)
15	4	4	3.27	4	4.00
127	7	7	6.06	7	7.00
1,023	10	10	9.01	10	10.00
1,000	10	10	8.99	10	9.99
10,000	14	14	12.36	14	13.37
100,000	17	17	15.68	17	16.69
1,000,000	20	20	18.95	20	19.95
1,048,575	20	20	18.99	20	20.00

The observed maximum equals $\lfloor \log_2 n \rfloor + 1$ in every row, so the bound is tight. A successful search costs about one probe less than the maximum on average because half of the elements sit at the deepest level of the implicit search tree. An absent target usually needs the full depth. Searching one million sorted records needs at most 20 probes; a linear scan needs on average 500,000.

3. Q2: The Quality of Decomposition in Sorting

3.1 Shared group work

Quick Sort pivot strategy (stated). The pivot is the last element of the current sub-array. The pivot is removed, the remaining elements are split into less ($x < \text{pivot}$) and geq ($x \geq \text{pivot}$) while keeping their relative order, and both parts are sorted recursively. Each partition of a sub-array of size m costs $m - 1$ comparisons. A stable list partition is used (not in-place) so that inputs with a known partition behaviour can be constructed; the partition sizes, the depth and the comparison counts are the same as for an in-place partition on the same pivot choices.

Merge Sort. The array is cut at $\lfloor n/2 \rfloor$, both halves are sorted recursively and merged, counting one comparison for each element placed while both halves are non-empty.

```
def quick(a, st, strategy, rng, d=1):
    st.depth = max(st.depth, d)
    if len(a) <= 1: return a
    i = pick(a, strategy, rng); p = a[i]        # last / first / median3 / random
    rest = a[:i] + a[i+1:]
    less = [x for x in rest if x <  p]
    geq  = [x for x in rest if x >= p]
    st.comps += len(rest); st.parts.append((len(a), len(less), len(geq)))
    L = quick(less, st, strategy, rng, d+1) if less else []
    R = quick(geq,  st, strategy, rng, d+1) if geq  else []
    return L + [p] + R

def merge_sort(a, st, d=1):
    st.depth = max(st.depth, d)
    n = len(a)
    if n <= 1: return a
    L = merge_sort(a[:n//2], st, d+1); R = merge_sort(a[n//2:], st, d+1)
    out = []; i = j = 0
    while i < len(L) and j < len(R):
        st.comps += 1
        if L[i] <= R[j]: out.append(L[i]); i += 1
        else:            out.append(R[j]); j += 1
    return out + L[i:] + R[j:]
```

3.2 Student A: Quick Sort investigation

Input that gives balanced partitions. With the last element as pivot, the input must end with the median of its values, and both halves before it must themselves follow the same rule. build_balanced(lo, hi) returns build(lo, m−1) + build(m+1, hi) + [m] with m the middle value. Because the partition is stable, less is exactly the first block and geq the second, so every pivot is a median.

Input that gives severely unbalanced partitions. An already sorted (ascending or descending) array: the last element is the maximum (or minimum) of its sub-array, so every partition is (m−1 : 0) or (0 : m−1). A 90:10 input was also constructed with the same recursive idea, placing the pivot at 10% of the value range.

Recorded partition sizes (n = 4095, pivot = last). Each triple is (size, left, right):

Partition no.	Balanced input	Ascending input
1	(4095, 2047, 2047)	(4095, 4094, 0)
2	(2047, 1023, 1023)	(4094, 4093, 0)
3	(1023, 511, 511)	(4093, 4092, 0)
4	(511, 255, 255)	(4092, 4091, 0)
5	(255, 127, 127)	(4091, 4090, 0)
6	(127, 63, 63)	(4090, 4089, 0)

Table 3.1: Quick Sort at n = 4095 (comparisons, maximum recursion depth, first split, share of balanced partitions)

Input	Pivot	Comparisons	Max depth	Root split	Balanced partitions
Balanced (built)	last	40,962	12	2047:2047	100.0%
Random	last	55,800	26	1923:2171	23.5%
Ascending	last	8,382,465	4,095	4094:0	0.0%
Descending	last	8,382,465	4,095	0:4094	0.0%
90:10 (built)	last	87,377	63	409:3685	0.0%
Random	median-of-3	47,415	22	1923:2171	49.5%
Ascending	median-of-3	40,962	12	2047:2047	100.0%
Descending	median-of-3	40,962	12	2047:2047	100.0%
90:10 (built)	median-of-3	82,938	58	409:3685	44.9%
Random	random	53,273	29	2983:1111	23.1%
Ascending	random	56,985	27	206:3888	24.7%
Descending	random	58,708	28	3888:206	23.4%
90:10 (built)	random	59,001	27	197:3897	24.0%

How pivot selection shapes the decomposition. The pivot decides the sizes of both subproblems, and nothing else does. The last-element pivot made the sorted input degenerate (8,382,465 = 4095 × 4094 / 2 comparisons, depth 4,095) but behaved well on random data (55,800 comparisons, depth 26). Median-of-three repaired the sorted inputs completely (40,962 comparisons, depth 12) but not the constructed 90:10 input (depth 58), so it is a heuristic, not a guarantee. A random pivot has no input that is bad every time: ascending, descending and 90:10 inputs all gave 53,000 to 59,000 comparisons and depth 27 to 29.

Table 3.2: Growth with n, pivot = last element

n	Balanced: comps / depth	Ascending: comps / depth	Random: comps / depth	Merge Sort: comps / depth	n $\log_2 n$	n(n−1)/2
127	642 / 7	8,001 / 127	825 / 13	721 / 8	888	8,001
255	1,538 / 8	32,385 / 255	2,080 / 18	1,722 / 9	2,039	32,385
511	3,586 / 9	130,305 / 511	4,732 / 19	3,941 / 10	4,598	130,305
1,023	8,194 / 10	522,753 / 1,023	11,379 / 23	8,931 / 11	10,229	522,753
2,047	18,434 / 11	2,094,081 / 2,047	28,691 / 28	19,973 / 12	22,516	2,094,081
4,095	40,962 / 12	8,382,465 / 4,095	60,263 / 30	43,970 / 13	49,139	8,382,465

The ascending column equals n(n−1)/2 exactly, and each doubling of n multiplies it by about 4. The balanced column grows like n $\log_2$ n and its depth grows by 1 per doubling.

3.3 Student B: comparative divide-and-conquer analysis

Merge Sort's decomposition. Merge Sort splits by position, never by value: a sub-array of size m always becomes ⌊m/2⌋ and ⌈m/2⌉. Its recursion tree therefore depends only on n. At n = 4095 the depth was 13 = ⌈$\log_2$ n⌉ + 1 and the root split was 2047:2048 for random, ascending and descending input alike. The work is shifted to the combine step, which costs at most m − 1 comparisons per node, so

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &\Rightarrow\ T(n) = \Theta(n\log n) \quad \text{for every input} \end{aligned}$$

The number of comparisons does vary with the data (44,029 random against about 24,500 sorted at n = 4095), but only between roughly ½ n $\log_2$ n and n $\log_2$ n.

Comparison with Quick Sort. Quick Sort splits by value. The subproblem sizes are decided by where the pivot falls in the sorted order, so the tree shape depends on the data and the pivot rule. Its combine step is free (concatenation), because the work was done during division.

Why Merge Sort is more predictable. The split point is chosen without looking at the data, so no input can change the tree. For Quick Sort the same input can produce depth 12 or depth 4,095 depending only on the pivot rule (Table 3.1).

Partition quality and tree height. Let every partition put a fraction α of the elements on the smaller side. Then the larger side is at most (1 − α)m, and the depth is at most log base 1/(1−α) of n. Work per level is at most n comparisons, so

$$\begin{aligned} \text{comparisons} &\le n \cdot \log_{1/(1-\alpha)} n \\ &= \Theta(n\log n) \quad \text{for any constant } \alpha > 0 \end{aligned}$$

A 50:50 split gives depth $\log_2 n$. A 90:10 split gives depth about log base 10/9 of n, which is 6.6 times $\log_2 n$ in theory. Measured on the constructed 90:10 input:

Table 3.3: Effect of a 90:10 split compared with 50:50 (pivot = last)

n	Depth 90:10	Depth 50:50	Comparisons 90:10	Comparisons 50:50	Ratio
255	37	8	3,317	1,538	2.16
1,023	50	10	17,434	8,194	2.13
4,095	63	12	87,377	40,962	2.13
16,383	76	14	419,055	196,610	2.13

A constant 90:10 split keeps the algorithm at $\Theta(n \log n)$; it only enlarges the constant by about 2.13 (the reciprocal of the entropy 0.469 bits of a 90:10 split) and adds about 13 levels each time n is multiplied by 4 (observed: 37, 50, 63, 76) instead of 2 levels for a 50:50 split. Measured depths are smaller than the theoretical log base 10/9 of n (63 against 79 at n = 4095) because small sub-arrays end the recursion earlier.

How repeated imbalance leads to $O(n^2)$. If the smaller side is empty at every step ($\alpha \to 0$), each partition removes only the pivot:

$$T(n) = T(n-1) + (n-1), \quad T(1) = 0$$
$$\Rightarrow T(n) = n(n-1)/2 = \Theta(n^2)$$

The recursion tree becomes a chain of height n, which is exactly the ascending-input row in Table 3.2 (8,382,465 = 4095 × 4094 / 2). The recursion stack also grows to n frames, which is a memory problem as well as a time problem. Merge Sort's recurrence never degenerates this way because its split is fixed at n/2.

Measured time at n = 100,000 (random input, median of 3 runs). Quick Sort with a random pivot took 290.9 ms (1,973,870 comparisons, depth 38) and Merge Sort took 332.3 ms (1,536,533 comparisons, depth 18). The expected comparison count for Quick Sort on random input is $2(n+1)H_n - 4n \approx 2{,}018{,}000$ (observed is 2.2% lower), against about 1,560,964 for Merge Sort ($n \log_2 n - n$). Quick Sort does about 28% more comparisons here, yet the two times are within 15% of each other in Python.

3.4 Joint design question

If the input characteristics are not known in advance, what information is needed before choosing between the two strategies?

1. **How sorted the data already is, and whether it can be adversarial.** Sorted, reverse-sorted or crafted input is the case in which a deterministic pivot rule fails (Table 3.1). If the data comes from untrusted users, a fixed pivot rule is a risk.
2. **Whether a worst-case guarantee is required.** Real-time or service-level limits favour Merge Sort, which is $\Theta(n \log n)$ for every input. If only average speed matters, Quick Sort with a random or median-of-three pivot is acceptable.
3. **Available memory.** Merge Sort needs $\Theta(n)$ extra space for arrays; Quick Sort sorts in place with $O(\log n)$ stack when the smaller side is recursed first.
4. **Stability.** If equal keys must keep their order (for example sorting by a second key after a first sort), Merge Sort is stable and standard Quick Sort is not.
5. **Duplicate keys.** Many equal keys degrade a two-way partition; a three-way partition or Merge Sort avoids this.
6. **Data structure and location.** Linked lists and data on disk suit Merge Sort (sequential access, no random access needed); in-memory arrays suit Quick Sort's cache behaviour.
7. **Cost of a comparison versus a move.** Merge Sort makes fewer comparisons, which matters when comparing keys is expensive.

Decision. With nothing known, use Merge Sort (or a hybrid such as introsort, which switches to Heap Sort when the depth exceeds $2 \log_2 n$) because the cost of not knowing is bounded. Choose randomised Quick Sort only if memory is tight and the data is known not to be hostile.

4. Q3: Breaking the Quadratic Barrier with Karatsuba Multiplication

4.1 Shared group work

Two n-digit integers are stored as little-endian digit lists (base 10) padded to a power of two. The cost measure is the number of single-digit multiplications.

Standard multiplication multiplies every digit of x by every digit of y: $n \times n = n^2$ single-digit multiplications.

Karatsuba is implemented on the same digit lists. The sums $(x_0 + x_1)$ are kept as un-carried coefficient lists, so each of the three subproblems has exactly n/2 coefficients and carries are resolved once at the end. Every result was compared with Python's built-in integer product on 300 random pairs plus edge cases (zeros, all nines, powers of ten) and all matched.

```
def school(a, b):
    res = [0] * (len(a) + len(b) - 1)
    for i, ai in enumerate(a):
        for j, bj in enumerate(b):
            res[i + j] += ai * bj
    C.mult += len(a) * len(b)
    return res

def karatsuba(a, b, cutoff=1):
    n = len(a)
    if n <= cutoff: return school(a, b)
    h = n // 2
    a0, a1, b0, b1 = a[:h], a[h:], b[:h], b[h:]
    z0 = karatsuba(a0, b0, cutoff)       # low  x low
    z2 = karatsuba(a1, b1, cutoff)       # high x high
    sa = [x + y for x, y in zip(a0, a1)]
    sb = [x + y for x, y in zip(b0, b1)]
    z1 = karatsuba(sa, sb, cutoff)       # (a0+a1)(b0+b1)
    res = [0] * (2 * n - 1)
    for i, v in enumerate(z0): res[i] += v
    for i in range(len(z1)):  res[i + h] += z1[i] - z0[i] - z2[i]
    for i, v in enumerate(z2): res[i + 2 * h] += v
    return res
```

4.2 Student A: algorithmic derivation

Decomposition. Let m = n/2 and B = 10. Write each integer as a high half and a low half:

$$x = x_1 B^m + x_0, \qquad y = y_1 B^m + y_0$$

Subproblems. The product is

$$xy = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

Computed directly, this needs four half-size products (x_1y_1, x_1y_0, x_0y_1, x_0y_0), giving $T(n) = 4T(n/2) + \Theta(n) = \Theta(n^2)$, no better than the standard method.

Why Karatsuba needs fewer recursive multiplications. The middle coefficient is not needed as two separate products, only as their sum. Since

$$(x_1 + x_0)(y_1 + y_0) = x_1 y_1 + (x_1 y_0 + x_0 y_1) + x_0 y_0$$

the sum of the cross products equals $(x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0$. Only three half-size multiplications remain: $z_2 = x_1y_1$, $z_0 = x_0y_0$ and $z_1 = (x_1 + x_0)(y_1 + y_0)$. The trade is one multiplication for a few additions and subtractions of $\Theta(n)$ cost.

Recurrence. Three subproblems of size n/2 plus linear work for the additions, subtractions and shifts:

$$T(n) = 3\,T(n/2) + c\,n, \qquad T(1) = 1$$

Solution (Master Theorem). Here a = 3, b = 2 and f(n) = cn. The critical exponent is $\log_2 3 \approx 1.585$, so $n^{\log_2 3} = n^{1.585}$. Because $f(n) = O(n^{1.585-\varepsilon})$ with $\varepsilon \approx 0.58$, Case 1 applies:

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585})$$

Solution (unrolling, $n = 2^k$). Level i has 3^i subproblems of size $n/2^i$, costing $c \cdot n \cdot (3/2)^i$ in total. The sum is a geometric series dominated by its last term, which is $3^k = 3^{\log_2 n} = n^{\log_2 3}$:

$$\begin{aligned} T(n) &= 3^k + c\,n \sum_{i=0}^{k-1} (3/2)^i \\ &= \Theta\left(3^k\right) = \Theta\left(n^{\log_2 3}\right) \end{aligned}$$

Evidence. With cutoff 1 digit the measured number of multiplications is exactly 3^k for $n = 2^k$ (Table 4.1), and a fit of log(count) against log(n) gives the exponent 1.5850, equal to $\log_2 3$, against 2 for the standard method.

Table 4.1: Single-digit multiplications and additions

n (digits)	Standard n^2	Karatsuba	3^k	Karatsuba / standard	Karatsuba additions
2	4	3	3	0.7500	7
4	16	9	9	0.5625	40
8	64	27	27	0.4219	163
16	256	81	81	0.3164	580
64	4,096	729	729	0.1780	6,160
256	65,536	6,561	6,561	0.1001	59,260
1,024	1,048,576	59,049	59,049	0.0563	548,680
2,048	4,194,304	177,147	177,147	0.0422	1,658,323

4.3 Student B: crossover investigation

Experimental design. Input sizes n = 4, 8, ..., 8,192 digits (powers of two), random digits (seed 2401). Each size was timed with 15 repetitions up to 64 digits, 7 up to 512, 3 up to 4,096, and

1 at 8,192 (the longest run is 5 s); the median is reported. Karatsuba was run with recursion down to 1 digit and with school-book multiplication below cutoffs of 8, 16, 32 and 64 digits.

Table 4.2: Time in ms against digits n

n	Standard	Karatsuba, cutoff 1	Cutoff 8	Cutoff 16	Cutoff 32	Cutoff 64
4	0.001	0.013	0.001	0.001	0.001	0.001
8	0.003	0.040	0.004	0.003	0.003	0.003
16	0.011	0.119	0.016	0.011	0.012	0.012
32	0.043	0.376	0.053	0.042	0.043	0.040
64	0.176	1.191	0.186	0.148	0.152	0.178
128	0.755	3.647	0.620	0.514	0.505	0.593
256	3.212	11.038	1.859	1.576	1.567	1.795
512	15.002	33.704	6.364	5.129	5.122	5.853
1,024	74.176	101.856	21.517	22.604	17.435	19.174
2,048	285.022	300.526	59.582	50.310	50.769	62.540
4,096	1,180.568	1,030.881	222.200	160.888	156.183	177.388
8,192	4,870.034	2,944.212	590.144	492.211	528.745	559.223

Where a cutoff is not smaller than n, the algorithm is the standard method and the two columns agree up to timing noise.

Crossover range.

- **Pure Karatsuba (recursing to single digits)** first becomes faster at 4,096 digits (1,030.9 ms against 1,180.6 ms) and stays faster. At 2,048 digits the two are level (300.5 ms against 285.0 ms). The crossover is therefore between 2,048 and 4,096 digits.
- **Hybrid Karatsuba (standard multiplication below a cutoff of 16 to 32 digits)** reaches about 15% faster at 64 digits (0.148 ms against 0.176 ms), 33% faster at 128, and about 10 times faster at 8,192 digits (492 ms against 4,870 ms). The practical crossover for a tuned implementation is about 32 to 64 digits.

Consistency with the model. From n = 1,024 the pure Karatsuba time is 101.9 ms / 3^{10} = 0.00173 ms per base multiplication and the standard time is 74.2 ms / $1{,}024^2$ = 0.0000707 ms per multiplication, a constant-factor ratio of about 24. Setting $\kappa \cdot n^{1.585} = s \cdot n^2$ gives $n^* = 24.4^{1/0.415} \approx 2{,}200$ digits, which lies inside the observed range of 2,048 to 4,096.

Why Karatsuba is worse for small inputs. The standard method is a tight double loop. Karatsuba pays, at every node of the recursion tree, for slicing four lists, forming two sum lists, a three-way combination and a function call. Table 4.3 shows that at n = 1,024 with a cutoff of

1 digit there are 88,573 calls and 548,680 additions for 59,049 multiplications: that is about 9.3 additions and 1.5 calls for every multiplication performed, and each of these costs more in Python than one multiply-accumulate step of the standard loop.

Table 4.3: Overhead against saved multiplications (n = 1,024)

Cutoff (digits)	Multiplications	Additions	Recursive calls
1	59,049	548,680	88,573
2	78,732	410,899	29,524
4	104,976	286,240	9,841
8	139,968	192,199	3,280
16	186,624	125,860	1,093
32	248,832	80,419	364
64	331,776	49,720	121
128	442,368	29,119	40
1,024 (standard)	1,048,576	0	1

Raising the cutoff trades multiplications for fewer calls and additions. The fastest setting in Table 4.2 (cutoff 16 to 32) sits between the extremes.

4.4 Joint conclusion

Why does better asymptotic complexity not automatically give better performance at every size?

Asymptotic notation hides the constant factor and the lower-order terms, and it describes behaviour only as n grows without bound. Karatsuba's true cost is of the form $\kappa \cdot n^{1.585}$ + (lower terms), against $s \cdot n^2$ for the standard method, with κ about 24 times larger than s in this implementation. The n^2 curve is below the $n^{1.585}$ curve until $n^{0.415}$ exceeds κ/s, which happens near 2,000 digits here. Below that point the extra additions, list copies and calls cost more than the multiplications they remove. The crossover moves with the language, the base-case routine, the word size and the cache, so it must be measured, and the practical remedy is a hybrid that uses the standard method below a tuned threshold.

5. Q4: The Reality of Strassen's Matrix Multiplication

5.1 Shared group work

Two implementations were written for n × n matrices: a conventional multiplication (three nested loops, written as row-by-column dot products), and Strassen's algorithm that splits each

matrix into four n/2 × n/2 blocks, recurses with seven block products and falls back to the conventional routine below a cutoff size. Matrices whose size is not a power of two are padded with zeros. Two variants were built: pure Python (lists of lists, integer entries, instrumented with counters) and numpy (float64, A @ B from BLAS as the base case).

Correctness. Strassen with padding returned exactly the conventional product for n = 1 to 13, 17, 31, 33 and 50 with cutoffs 1 and 2 (integer entries, exact comparison). The numpy version differed from BLAS by a maximum relative error of 4.5×10^{-15} at n = 512.

```
def strassen(A, B, cutoff=1):
    n = len(A)
    if n <= cutoff:
        C.mult += n**3; C.add += n*n*(n-1)
        return conv(A, B)
    A11, A12, A21, A22 = quad(A); B11, B12, B21, B22 = quad(B)
    M1 = strassen(madd(A11, A22), madd(B11, B22), cutoff)
    M2 = strassen(madd(A21, A22), B11, cutoff)
    M3 = strassen(A11, msub(B12, B22), cutoff)
    M4 = strassen(A22, msub(B21, B11), cutoff)
    M5 = strassen(madd(A11, A12), B22, cutoff)
    M6 = strassen(msub(A21, A11), madd(B11, B12), cutoff)
    M7 = strassen(msub(A12, A22), madd(B21, B22), cutoff)
    h = n // 2; C.add += 18 * h * h
    C11 = madd(msub(madd(M1, M4), M5), M7);  C12 = madd(M3, M5)
    C21 = madd(M2, M4);                      C22 = madd(msub(madd(M1, M3), M2), M6)
    return join(C11, C12, C21, C22)
```

5.2 Student B: algebraic design and correctness

Conventional block decomposition. Partition A, B and C into four n/2 × n/2 blocks. Then

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22},$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

The eight standard block multiplications are $A_{11}B_{11}$, $A_{12}B_{21}$, $A_{11}B_{12}$, $A_{12}B_{22}$, $A_{21}B_{11}$, $A_{22}B_{21}$, $A_{21}B_{12}$ and $A_{22}B_{22}$. The recurrence is $T(n) = 8T(n/2) + \Theta(n^2)$, which solves to $\Theta(n^3)$ (Master Theorem Case 1, since $n^{\log_2 8} = n^3$). Block recursion alone gives no gain over three nested loops.

The transformation to seven multiplications. Strassen forms seven products from sums and differences of blocks:

$$
\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), & M_2 &= (A_{21} + A_{22})B_{11}, \\
M_3 &= A_{11}(B_{12} - B_{22}), & M_4 &= A_{22}(B_{21} - B_{11}), \\
M_5 &= (A_{11} + A_{12})B_{22}, & M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\
M_7 & & &= (A_{12} - A_{22})(B_{21} + B_{22})
\end{aligned}
$$

$$
\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\
C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6
\end{aligned}
$$

Why the reconstruction is correct. Expanding each block (block products do not commute, so the order of factors is kept):

- $C_{12} = M_3 + M_5 = A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{22} + A_{12}B_{22} = A_{11}B_{12} + A_{12}B_{22}$.
- $C_{21} = M_2 + M_4 = A_{21}B_{11} + A_{22}B_{11} + A_{22}B_{21} - A_{22}B_{11} = A_{21}B_{11} + A_{22}B_{21}$.
- $C_{11} = M_1 + M_4 - M_5 + M_7$. M_1 contributes $A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22}$; M_4 contributes $A_{22}B_{21} - A_{22}B_{11}$; $-M_5$ contributes $-A_{11}B_{22} - A_{12}B_{22}$; M_7 contributes $A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22}$. Every term cancels except $A_{11}B_{11} + A_{12}B_{21}$.
- $C_{22} = M_1 - M_2 + M_3 + M_6$. After cancelling $A_{11}B_{11}$, $A_{11}B_{22}$, $A_{22}B_{11}$, $A_{21}B_{11}$ and $A_{11}B_{12}$ in pairs, only $A_{22}B_{22} + A_{21}B_{12}$ is left.

The transformation spends 18 block additions or subtractions (10 on the inputs, 8 on the outputs) to save one block multiplication:

$$
\begin{aligned}
T(n) &= 7T(n/2) + 18/4\, n^2 \\
&\Rightarrow\ T(n) = \Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807})
\end{aligned}
$$

Because $n^{\log_2 7} = n^{2.807}$ grows faster than n^2, the Master Theorem's Case 1 applies. With recursion to 1×1 blocks the counts are exactly 7^k multiplications and $6(7^k - 4^k)$ additions (Table 5.1).

Padding. The recursion needs even sizes at every level. A matrix of size n is embedded in the top-left corner of an $N \times N$ matrix, $N = 2^{\lceil \log_2 n \rceil}$, with zeros elsewhere. Since [A 0; 0 0] · [B 0; 0 0] = [AB 0; 0 0], the product of the padded matrices contains AB in its top-left $n \times n$ block, and the rest is discarded. The cost is that N can be almost 2n, so up to 75% of the padded matrix is zeros and the multiplication count can rise by up to 7 times ($2^{2.807}$) compared with the true size. Table 5.5 quantifies this.

5.3 Student A: experimental and engineering analysis

Experiment design. (1) Operation counts for n = 2 to 128 with recursion to 1×1, checked against the formulas. (2) Analytic totals (multiplications plus additions) for n up to 8,192,

comparing the best cutoff with the conventional method. (3) Wall-clock time for pure Python for n = 16 to 512, cutoffs 8 to 64, with 7, 5, 3 or 1 repetitions depending on n (median). (4) numpy time for n = 128 to 2,048 with cutoffs 64, 128 and 256 (7 or 3 repetitions), plus peak extra memory from tracemalloc. (5) Non-power-of-two sizes for the padding cost.

Table 5.1: Counts with recursion to 1 × 1 (instrumented)

n	Conventional multiplications	Strassen multiplications ($= 7^k$)	Ratio	Strassen additions = $6(7^k - 4^k)$	Conventional total ops	Strassen total ops
2	8	7	0.875	18	12	25
4	64	49	0.766	198	112	247
8	512	343	0.670	1,674	960	2,017
16	4,096	2,401	0.586	12,870	7,936	15,271
32	32,768	16,807	0.513	94,698	64,512	111,505
64	262,144	117,649	0.449	681,318	520,192	798,967
128	2,097,152	823,543	0.393	4,842,954	4,177,920	5,666,497

The multiplication count is lower from n = 2 onward, and the instrumented counts match 7^k and $6(7^k - 4^k)$ exactly. But the additions raise the total operation count above the conventional method at every size in this table.

Table 5.2: Total arithmetic operations (multiplications + additions), analytic

n	Conventional	Strassen, cutoff 1	Strassen, best cutoff (8)	Best / conventional
16	7,936	15,271	7,872	0.992
64	520,192	798,967	436,416	0.839
256	33,488,896	39,960,391	22,195,392	0.663
512	268,173,312	280,902,385	156,547,392	0.584
1,024	2,146,435,072	1,971,035,287	1,100,550,336	0.513
2,048	17,175,674,880	13,816,121,377	7,722,726,720	0.450
8,192	1,099,444,518,912	677,820,419,665	379,244,081,472	0.345

Counting additions as well, full recursion to 1 × 1 overtakes the conventional method only at n = 1,024, while stopping the recursion at 8 × 8 blocks gives fewer total operations from n = 16.

Table 5.3: Pure Python time in ms

n	Conventional	Strassen c = 8	c = 16	c = 32	c = 64	Best / conventional
16	0.17	0.36	-	-	-	2.06
32	1.26	3.02	1.70	-	-	1.34
64	15.28	31.25	14.07	13.72	-	0.90
128	77.56	172.40	108.42	85.22	73.92	0.95
256	587.40	1224.30	858.93	650.54	546.67	0.93
512	4673.53	9204.82	6135.78	4866.59	4288.24	0.92

A dash means the cutoff is not smaller than n. In pure Python Strassen is slower below n = 64 and 5 to 10% faster from n = 64 upward with a cutoff of 32 to 64, so the practical crossover is in the range 64 to 128. The gain is small because Python's list additions cost far more per element than the conventional dot-product loop, which runs inside the interpreter's sum and map built-ins.

Table 5.4: numpy (BLAS base case), float64, one thread

n	Conventional ms	Conventional GFLOP/s	Strassen c = 64	c = 128	c = 256	Best / conventional	Peak extra memory, conventional (MB)	Peak extra memory, Strassen c = 128 (MB)
128	0.09	45.0	0.20	-	-	2.10	0.1	0.1
256	0.71	47.4	1.91	1.15	-	1.62	0.5	1.7
512	5.17	51.9	16.31	11.80	7.35	1.42	2.1	6.3
1,024	40.86	52.6	122.16	92.46	64.89	1.59	8.4	25.2
2,048	297.15	57.8	1050.17	657.36	565.02	1.90	33.6	100.7

Against an optimised library kernel, Strassen was slower at every size tested, by a factor of 1.4 to 2.1. No crossover was reached up to n = 2,048.

Overhead analysis.

- **Additions.** Each level adds 18 block additions, $\Theta(n^2)$ work each, which are memory-bound operations that run far below the speed of the matrix-multiplication kernel (Table 5.1 shows 4.8 million additions against 0.8 million multiplications at n = 128).
- **Recursion.** Calls and slicing (quad) copy sub-matrices; at n = 512 with cutoff 8 the pure Python version took 9,205 ms against 4,674 ms for the conventional version because $7^6 = 117{,}649$ small base products were created.
- **Padding.** See Table 5.5.

- **Memory and cache.** The conventional kernel reached 45 to 58 GFLOP/s and its rate rose with n because larger blocks reuse the cache better. Strassen allocates temporaries for seven products and ten sums at each level; the peak extra memory was three times the output size at n = 2,048 (100.7 MB against 33.6 MB). Larger cutoffs were faster in every row of Table 5.4 (cutoff 256 best at n = 512 to 2,048), which shows that the additions and temporaries, not the multiplications, dominate the time: the more of the work is left to the cache-blocked kernel, the better.

Table 5.5: Padding cost for non-power-of-two sizes (multiplications)

n	Padded N	Conventional n^3	Strassen, cutoff 16, size N	Strassen / conventional	Zero fill
64	64	262,144	200,704	0.77	0.0%
65	128	274,625	1,404,928	5.12	74.2%
100	128	1,000,000	1,404,928	1.40	39.0%
128	128	2,097,152	1,404,928	0.67	0.0%
129	256	2,146,689	9,834,496	4.58	74.6%
300	512	27,000,000	68,841,472	2.55	65.7%
500	512	125,000,000	68,841,472	0.55	4.6%
512	512	134,217,728	68,841,472	0.51	0.0%

The count for n = 100 was also obtained by instrumenting the padded run (1,404,928) and agrees with the formula. A size just above a power of two (65, 129) makes Strassen worse than the conventional method by a factor of four to five, which gives the cost of padding in the worst case.

5.4 Joint engineering decision

When would we use conventional multiplication despite Strassen's better asymptotic complexity?

Conventional multiplication is the better choice when:

1. **The matrices are small or medium.** In these experiments Strassen won (by 5 to 10%) only above n of about 64 to 128 in pure Python, and not at all up to n = 2,048 against BLAS.
2. **An optimised library or hardware kernel is available** (BLAS, GPU, SIMD). These run the conventional algorithm near peak speed, while Strassen's additions run at memory speed.

3. **The size is not close to a power of two,** or is just above one (n = 65 or 129), so that padding wastes most of the work, unless a peeling or dynamic-padding scheme is used.
4. **Memory is limited.** The temporaries cost about three times the output size in our runs.
5. **Numerical accuracy matters.** Strassen's error bound is weaker (normwise rather than componentwise), which is unacceptable for some floating-point applications. Exact integer arithmetic does not have this issue.
6. **The matrices are sparse or structured,** where specialised kernels do far fewer than n^3 operations.

Strassen is justified for very large dense matrices (several thousand rows or more) when the base case is a tuned kernel with a measured cutoff, when memory is available, and when the accuracy requirement is relaxed or the arithmetic is exact.

6. Q5: Upgrading Closest Pair Using Divide and Conquer

6.1 Shared group work

Brute force (from CO2). Compare every pair of the n points and keep the smallest distance: $n(n-1)/2$ distance calculations, $\Theta(n^2)$.

Divide and conquer design. The cost measure is the number of squared-distance evaluations (square roots are avoided, since squared distances preserve the order). Both algorithms were given identical point sets, and the D&C distance was compared with the brute-force distance on 300 random cases, including duplicated points and ties; all matched.

```
def closest_dc(Px):        # Px: points sorted by x
    n = len(Px)
    if n <= 3:        # base case: brute force on <= 3 points
        b = brute(Px); return b[0], b[1], b[2], sorted(Px, key=lambda p: p[1])
    mid = n // 2; xm = Px[mid][0]        # dividing line x = xm
    dl, pl, ql, Yl = closest_dc(Px[:mid])
    dr, pr, qr, Yr = closest_dc(Px[mid:])
    best = (dl, pl, ql) if dl <= dr else (dr, pr, qr)
    Y = merge_by_y(Yl, Yr)        # O(n) merge, as in merge sort
    d = best[0]
    strip = [p for p in Y if (p[0] - xm) ** 2 < d]
    for a in range(len(strip)):
        b = a + 1
        while b < len(strip) and (strip[b][1] - strip[a][1]) ** 2 < d:
            v = d2(strip[a], strip[b])
            if v < d: d = v; best = (v, strip[a], strip[b])
            b += 1
    return best[0], best[1], best[2], Y
```

6.2 Student A: decomposition design

- **How the points are organised.** The points are sorted once by x. Each recursive call also returns its points sorted by y, which is built by merging the two sorted halves, so the strip never has to be re-sorted.
- **How the problem is divided.** The sorted array is cut at the middle index. The x-coordinate of the middle point defines a vertical line; ⌊n/2⌋ points lie on the left and ⌈n/2⌉ on the right. A sub-problem with at most 3 points is solved by brute force (at most 3 distance calculations).
- **What each recursive call returns.** The smallest squared distance in its half, the pair that achieves it, and the half's points sorted by y.
- **Combine step.** Let δ^2 be the smaller of the two returned distances. The only pairs not yet examined have one point on each side of the line. Merge the two y-sorted lists, keep the points whose x-distance to the line is below δ (the strip), and for each strip point compare it with the points that follow it in y-order while their y-distance is below δ. Update δ whenever a closer pair is found.

- **Why the boundary needs special treatment.** The closest pair in the whole set may have one point in each half. Neither recursive call has seen such a pair, so the answer $\min(\delta_{left}, \delta_{right})$ can be wrong. A cross pair can only beat δ if both points are within δ of the dividing line, which is why the strip is examined.

Recurrence. Two half-size problems, plus a linear merge and a linear strip scan:

$$T(n) = 2T(n/2) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n\log n)$$

The initial sort by x is also Θ(n log n), so the whole algorithm is Θ(n log n) against $\Theta(n^2)$ for brute force.

6.3 Student B: comparative evidence

Datasets. Uniform random points in $[0, 10^6]^2$ (seed 2401) for the main comparison, with n from 10 to 1,000,000. Brute force was executed up to n = 5,000; for larger n its count is the exact value n(n − 1)/2 and its time is not measured, because the run would not finish in a reasonable time. Two further datasets stress the strip: ten Gaussian clusters, and all points on a vertical line (x within ±0.001), which puts every point inside the strip at the top level.

Table 6.1: Distance calculations and execution time (uniform data)

n	Brute-force calculations	D&C calculations	D&C / brute	Brute ms	D&C ms
10	45	11	0.244	0.02	0.02
50	1,225	57	0.0465	0.27	0.14
100	4,950	129	0.0261	1.06	0.26
200	19,900	259	0.0130	4.54	0.55
500	124,750	569	0.00456	28.26	1.83
1,000	499,500	1,132	0.00227	151.02	3.96
2,000	1,999,000	2,299	0.00115	505.68	8.28
5,000	12,497,500	6,429	0.00051	2,853.45	21.59
10,000	49,995,000	13,008	0.00026	not run	48.30
100,000	4,999,950,000	142,498	0.00003	not run	791.15
1,000,000	499,999,500,000	1,218,412	0.000002	not run	13,109.82

Growth. Each doubling of n multiplies the brute-force count by about 4 (1,999,000 at n = 2,000 against 499,500 at n = 1,000) but multiplies the D&C count by about 2 (2,299 against 1,132). At n = 1,000,000 brute force needs 5×10^{11} calculations. At the measured rate of 0.23 µs per calculation (n = 5,000: 2,853 ms for 12.5 million), that would be roughly 32 hours, extrapolated and not measured, while D&C finished in 13.1 s including sorting and merging.

Table 6.2: Strip behaviour of the D&C algorithm

Dataset	n	Strip calculations	Strip points (all levels)	Largest strip	Maximum candidates compared per strip point
Uniform	1,000	610	3,184	20	2
Uniform	100,000	46,094	478,036	109	2
Clustered	1,000	550	2,433	16	2
Clustered	100,000	44,149	386,273	97	2
Vertical line	1,000	431	8,928	1,000	1
Vertical line	10,000	3,448	120,000	10,000	1
Vertical line	100,000	28,227	1,485,860	79,076	1

Where D&C becomes practically advantageous. The distance count is lower from n = 4 onward (4 against 6). Wall-clock time also includes sorting, merging and recursion, so it takes longer to pay off. Median of 9 runs:

Table 6.3: Crossover in wall-clock time (uniform data)

n	Brute ms	D&C ms	D&C / brute
8	0.0073	0.0153	2.11
12	0.0154	0.0180	1.17
16	0.0264	0.0317	1.20
20	0.0402	0.0377	0.94
30	0.0907	0.0671	0.74
50	0.2525	0.1106	0.44
100	1.0294	0.2469	0.24
200	4.1647	0.5535	0.13

D&C becomes faster at about 20 points (between 16 and 20) and stays faster from there. It is already 4 times faster at n = 100 and 38 times faster at n = 1,000 (151.0 ms against 4.0 ms). A practical implementation would use brute force below roughly 16 to 20 points as its base case.

6.4 Cross-understanding

Student B explains: why does the strip stage not compare each point with every other point?

Only points within δ of the dividing line can form a cross pair closer than δ, and for each of them only points whose y-coordinate lies within δ above it matter (the pair is examined once, from its lower point). Those candidates fit in a rectangle of width 2δ (δ on each side of the line)

and height δ. Each half of that rectangle is a δ × δ square, and the points inside one half-side are pairwise at least δ apart (δ is the minimum distance already found in that half), so at most 4 points fit in a δ × δ square, one per corner. The rectangle therefore holds at most 8 points, including the point being examined, so each strip point is compared with at most 7 others, not with the other n points. The strip scan is Θ(n), which keeps the combine step linear. The experiment agrees: even when all 10,000 points were in the strip (vertical line), no point was compared with more than 1 candidate, and in uniform data the maximum was 2.

Student A explains: what experimental evidence shows that D&C reduced the computational work?

The operation counts (Table 6.1), measured identically in both programs, give the evidence. For the same 5,000 points brute force performed 12,497,500 distance calculations and D&C performed 6,429 (0.05% of the work); at n = 1,000 the figures were 499,500 and 1,132. The growth also changed class: the brute-force count grows by 4 for each doubling of n and the D&C count by about 2, which is the difference between n^2 and n log n. The wall-clock time confirms that the saving is real after the overheads are paid: 2,853 ms against 21.6 ms at n = 5,000. Both algorithms returned the same minimum distance on every test.

7. Q6: Advanced Divide-and-Conquer Mini-Investigations

7.1 Student A: Median of Medians for the k-th smallest element

Algorithm. The k-th smallest element is found by choosing a pivot, partitioning around it, and recursing only into the side that contains position k. The choice of pivot is what Median of Medians (MoM) controls.

1. **Grouping.** Split the n elements into ⌈n/5⌉ groups of 5 (the last group may be smaller).
2. **Group medians.** Sort each group (insertion sort, at most 10 comparisons) and take its median.
3. **Pivot.** Find the median of those ⌈n/5⌉ medians by calling the same algorithm recursively. This value is the pivot.
4. **Partition and recurse.** Partition around the pivot into smaller, equal and larger elements and continue in the part that holds the k-th element.

```
def mom_select(a, k):
    n = len(a)
    if n <= 5: return insertion_sort_count(a)[k]
    groups = [a[i:i + 5] for i in range(0, n, 5)]        # 1. groups of 5
    meds = [insertion_sort_count(g)[len(g) // 2] for g in groups]        # 2. median of each group
    p = mom_select(meds, len(meds) // 2)        # 3. median of the medians
    less, eq, greater = partition3(a, p)        # 4. partition around p
    if k < len(less): return mom_select(less, k)
    if k < len(less) + eq: return p
    return mom_select(greater, k - len(less) - eq)
```

Why disastrous partitions cannot happen. At least half of the ⌈n/5⌉ group medians are ≤ the pivot. Each of those medians has two more elements of its group that are smaller or equal to it. Discounting the group of the pivot and the possibly incomplete last group, at least 3(⌈n/10⌉ − 2) ≥ 3n/10 − 6 elements are ≤ the pivot, and by symmetry at least 3n/10 − 6 are ≥ it. Therefore each side of the partition has at most 7n/10 + 6 elements, and the recursion never degenerates into removing a single element.

Worst-case guarantee. The recurrence is

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + 6) + c\,n$$

The first term finds the pivot, the second is the largest side that can remain, and cn is the grouping, sorting of small groups and partition. Assume T(m) ≤ a·m for m < n. Then T(n) ≤ a·n/5 + a(7n/10 + 6) + cn = a·n·(9/10) + 6a + cn, and this is at most a·n whenever a(n/10 − 6) ≥ cn. For n ≥ 120 we have n/10 − 6 ≥ n/20, so a = 20c is enough. Hence T(n) = O(n) for every input. The essential point is 1/5 + 7/10 = 9/10 < 1: the total size of the two recursive calls is smaller than n. With groups of 3 the sum would be 1/3 + 2/3 = 1 and the bound would become Θ(n log n).

Experiment against QuickSelect. Three strategies were compared: QuickSelect with the first element as the pivot (deterministic), QuickSelect with a random pivot (mean and maximum over 30 runs with different seeds), and MoM. Each partition step counts one comparison per element. Ascending input with k = n − 1 and descending input with k = 0 are the adversarial cases for the first-element pivot; the random input uses k = n/2. All results were checked against sorted(a)[k], and 500 randomised cases with duplicates also agreed.

Table 7.1: Comparisons to select the k-th smallest element

Input	n	k	QuickSelect, first pivot: comparisons (rounds)	QuickSelect, random pivot: mean / max of 30	MoM: comparisons (per element)	MoM: largest side seen
Ascending	1,000	999	500,500 (1,000)	1,859 / 2,955	5,250 (5.25)	0.552
Descending	1,000	0	500,500 (1,000)	1,859 / 2,955	8,929 (8.93)	0.540
Random	1,000	500	5,599 (18)	3,268 / 6,001	7,387 (7.39)	0.577
Ascending	2,000	1,999	2,001,000 (2,000)	3,721 / 5,852	10,872 (5.44)	0.540
Descending	2,000	0	2,001,000 (2,000)	3,721 / 5,852	18,368 (9.18)	0.533
Random	2,000	1,000	6,275 (15)	6,539 / 9,852	15,194 (7.60)	0.542
Ascending	4,000	3,999	8,002,000 (4,000)	7,433 / 11,716	22,298 (5.57)	0.561
Descending	4,000	0	8,002,000 (4,000)	7,433 / 11,716	37,496 (9.37)	0.540
Random	4,000	2,000	16,847 (19)	13,788 / 25,843	31,571 (7.89)	0.550
Ascending	8,000	7,999	32,004,000 (8,000)	14,876 / 23,451	45,251 (5.66)	0.542
Descending	8,000	0	32,004,000 (8,000)	14,876 / 23,451	76,090 (9.51)	0.540
Random	8,000	4,000	18,583 (13)	25,501 / 42,367	64,242 (8.03)	0.574

Deterministic poor-pivot strategy. With the first element as pivot, an ascending (or descending) input makes every pivot the minimum (or maximum) of the remaining elements, so each round removes one element: the number of rounds is n and the comparisons are $n(n + 1)/2$ (32,004,000 for n = 8,000, which is exactly 8,000 × 8,001 / 2). The same algorithm needs only 5,599 to 18,583 comparisons on random input, so the damage depends only on how the input is arranged. MoM stayed between 5.25 and 9.51 comparisons per element on all inputs, and its largest side never exceeded 0.577 of the array, below the guaranteed $0.7 + 6/n$.

Why randomisation reduces but does not eliminate poor partitions. A random pivot has no bad input, because no arrangement of the data can predict the choice. But the pivot rank is uniform, so a poor pivot still occurs by chance. For n = 10,000 and 2,000 random pivots, 20.1% left more than 90% of the array on the larger side and 1.95% left more than 99%; the mean larger side was 0.753. The MoM pivot left 0.506 on the larger side (bound 0.7006). The expected number of comparisons stays linear (about 1.9n to 3.2n in Table 7.1), but the worst-case run is not bounded, since n consecutive bad pivots would still give $\Theta(n^2)$ (with exponentially small probability), and the observed maximum of 30 runs was 1.6 times the mean. Only MoM turns the expected bound into a bound on every run.

Cost of the guarantee. For n = 100,000 random values and k = n/2, random QuickSelect took 13.4 ms and MoM 115.9 ms. MoM does 5 to 10 comparisons per element against about 2 to 3

for QuickSelect, so it is chosen when a worst-case bound is required, or as the fallback when QuickSelect's recursion becomes too deep.

7.2 Student B verification: why is pivot quality central to selection?

A selection algorithm discards the part of the array that cannot contain the k-th element, so its total work is the sum of the sizes of the successive sub-arrays. The pivot decides how large the surviving part is. A pivot near the median halves the problem ($n + n/2 + n/4 + ... = 2n$ comparisons); a pivot near an end removes almost nothing ($n + (n - 1) + ... = n^2/2$). The same algorithm therefore costs $\Theta(n)$ or $\Theta(n^2)$ depending only on pivot quality (Table 7.1). Median of Medians, random pivots and median-of-three differ only in how they bound the chance or the size of a bad pivot.

7.3 Student B: Meet-in-the-Middle subset sum

Problem. Given n positive integers $w_1 ... w_n$ and a target T, decide whether some subset sums to T.

How the problem is divided. The items are split into two halves of n/2 items each, which are independent of each other.

Subset results for both halves. For each half, all $2^{n/2}$ subset sums are generated by doubling: start from [0] and, for each item x, append (every current sum + x). The two lists are then sorted.

How the partial results are combined. A subset of the full set is a pair (subset of the left half, subset of the right half), so it sums to T exactly when some left sum a and some right sum b satisfy a + b = T. With both lists sorted, a two-pointer scan from the smallest left sum and the largest right sum finds such a pair or proves none exists in at most $2^{n/2}$ steps.

```
def mitm_subset(w, target):
    h = len(w) // 2
    L = sorted(all_sums(w[:h])); R = sorted(all_sums(w[h:]))       # 2 x 2^(n/2) sums
    i, j = 0, len(R) - 1
    while i < len(L) and j >= 0:       # two-pointer scan
        t = L[i] + R[j]
        if t == target: return True
        if t < target: i += 1
        else: j -= 1
    return False
```

Resulting search-space size. Instead of 2^n subsets, the algorithm generates $2 \cdot 2^{n/2}$ sums and scans at most $2^{n/2}$ pairs. The running time is $\Theta(2^{n/2} \cdot n)$ when sorting is included ($\Theta(2^{n/2})$ if the lists are built already sorted by merging). Counting steps in the experiment gave exactly 2^n for brute force and exactly $2 \cdot 2^{n/2}$ generated sums for MITM.

Additional memory. Both lists must be stored: $\Theta(2^{n/2})$ values, against O(n) for enumeration by Gray code. With 8-byte integers that is $2 \cdot 2^{n/2} \cdot 8$ bytes, which is 16.8 MB at n = 40, 537 MB at n = 50 and 17.2 GB at n = 60.

Table 7.2: Brute force against Meet-in-the-Middle (unreachable target, so the full search is forced)

n	Brute-force steps (2^n)	MITM sums generated	MITM scan steps	Brute-force time (ms)	MITM time (ms)	MITM peak memory in Python (MB)
10	1,024	64	32	0.2	0.0	-
18	262,144	1,024	512	38.2	0.2	-
20	1,048,576	2,048	1,024	156.3	0.8	0.1
22	4,194,304	4,096	2,048	877.6	0.8	-
24	16,777,216	8,192	4,096	2,694.3	1.6	0.4
28	268,435,456	32,768	16,384	est. 43,108	6.9	1.5
32	4,294,967,296	131,072	65,536	est. 689,731	29.8	6.0
36	68,719,476,736	524,288	262,144	est. 11,035,691	173.5	24.1
40	1,099,511,627,776	2,097,152	1,048,576	est. 176,571,057	1,033.0	-

Brute-force times above n = 24 are estimated as $2^n \times 0.161$ µs (the per-step time measured at n = 24); they were not run. MITM results were identical to brute force on 200 random instances with n = 8 to 18.

For n = 40 the estimate for brute force is about 49 hours, while MITM finished in 1.0 s. When the target is reachable, brute force also benefits from stopping early (n = 22: found after 888,323 of 4,194,304 subsets), while MITM needed 4,096 sums and 1,166 scan steps.

Comparison with 2^n enumeration and the time-memory trade-off. Peak memory grew by a factor of 4 for every 4 additional items (0.4, 1.5, 6.0, 24.1 MB), in line with $2^{n/2}$, while time shrank from exponential in n to exponential in n/2. MITM therefore spends memory to save time: the same machine can handle about twice as many items. The trade-off ends where memory runs out first. At n = 60 the time ($2^{30} \approx 10^9$ steps) is acceptable, but the lists need about

17 GB. If memory is much lower than that, the options are the Schroeppel-Shamir variant (same time, $O(2^{n/4})$ memory) or splitting the list into smaller pieces.

7.4 Student A verification: why can MITM reduce work despite the extra storage?

The two halves are independent, so the 2^n candidate subsets are the Cartesian product of two lists of $2^{n/2}$ partial results. Brute force walks through that product one element at a time. Storing and sorting the two lists costs $2^{n/2}$ space, and in return the question "is there a pair with a + b = T?" can be answered by looking up complements with a single scan instead of testing each pair. The stored partial results are reused many times (each left sum is matched against the whole right list at no extra cost), so memory buys the removal of repeated work. The saving is therefore exponential in n, while the cost is exponential in n/2 space.

8. Q7: Design Divide and Conquer from First Principles

Problem. The Day-4 problem is the *Maximum Subarray Sum*. Given an array a[0..n−1] of n integers, some of which may be negative, find the largest sum of a non-empty contiguous block a[i..j]. The straightforward baseline examines all n(n + 1)/2 blocks. Both students prepared a design before comparing notes, and both initial designs are retained below as they were first written.

8.1 Stage 1: Independent design proposals

Student A (proposal written before any discussion).

- **Decomposition:** split the array at the middle into a left half and a right half.
- **Base case:** a single element is its own maximum subarray.
- **Required subproblem information:** the maximum subarray sum of each half.
- **Combine strategy:** return the larger of the two half answers.

Student B (proposal written before any discussion).

- **Decomposition:** split the array at the middle into a left half and a right half.
- **Base case:** a single element is its own maximum subarray.
- **Required subproblem information:** the maximum subarray sum of each half.
- **Combine strategy:** return the largest of three values: the left answer, the right answer, and the best subarray that crosses the midpoint. The crossing value is found by scanning

leftwards from the midpoint and rightwards from the next element, and adding the best left extension to the best right extension.

8.2 Stage 2: Comparison of the two designs

The two proposals were compared point by point and then both were run on 3,000 random arrays (n = 1 to 60, values from −50 to 50, seed 2401) against brute force.

Table 8.1: Comparison of the two independent proposals

Aspect	Student A	Student B	Fate in the final design
Decomposition	Split at the middle	Split at the middle	Kept (both agree)
Base case	n = 1	n = 1	Kept (both agree)
Information returned	Best sum only	Best sum only	Insufficient; replaced by a four-value record
Combine	max(left, right)	max(left, right, crossing), crossing found by scanning	B's three-case structure kept; the scan replaced by constant-time formulas
Combine cost	Constant	Θ(n) per node	Constant cost kept
Wrong answers on 3,000 random arrays	2,756 (91.9%)	0	0

Where the designs agree. Both split by position at the middle, both use a single element as the base case, and both return only one number from each half. Neither design needs the subproblems to be split by value, so the recursion tree depends only on n.

Where they differ. Only the combine step differs. Student A assumes the best block lies entirely inside one half. Student B recognises a third possibility, a block that straddles the midpoint, and computes it explicitly.

Strengths and weaknesses. Student A's combine is cheap but incorrect: for [−5, 2, 3, 4, 5, −1, −9, 1] the proposal returns 5, while the correct answer is 14, the block 2 + 3 + 4 + 5. Student B's design is correct, but each combine rescans every element of its range, so a node of size m costs about m comparisons and the total is Θ(n log n). The rescan repeats work that the halves have already done: each half has already added up its own elements.

Missing cases and information. Student A misses the crossing case. Student B misses nothing in correctness, but the halves do not report enough. A crossing block is always the best suffix of the left half joined to the best prefix of the right half, so if each half also reports its best prefix and best suffix, no rescan is needed. Those two values in turn can only be updated in constant time if each half also reports its total. Both designs also have to handle all-negative

arrays, where the answer is the largest single element; the single-element base case does this, whereas an answer of 0 for an empty block would be wrong.

Ideas retained. From both: the middle split and the single-element base case. From Student B: the three-way case analysis (left, right, crossing). From Student A: the insistence on a constant-time combine. The new idea that came from the comparison is to return four values instead of one.

8.3 Stage 3: Joint final design

- **Decomposition:** split a[lo..hi] at mid = ⌊(lo + hi)/2⌋.
- **Subproblems:** the two ranges a[lo..mid] and a[mid + 1..hi], of sizes ⌈n/2⌉ and ⌊n/2⌋. They do not overlap and are independent.
- **Base case:** lo = hi. The single element x gives the record (x, x, x, x).
- **Information returned:** for its range, the total sum of all elements, the best prefix sum (non-empty), the best suffix sum (non-empty) and the best subarray sum (non-empty).
- **Combine operation:** with L and R denoting the records of the left and right halves,

$$\begin{aligned} \text{total} &= L.\text{total} + R.\text{total} \\ \text{prefix} &= \max(L.\text{prefix},\ L.\text{total} + R.\text{prefix}) \\ \text{suffix} &= \max(R.\text{suffix},\ R.\text{total} + L.\text{suffix}) \\ \text{best} &= \max(L.\text{best},\ R.\text{best},\ L.\text{suffix} + R.\text{prefix}) \end{aligned}$$

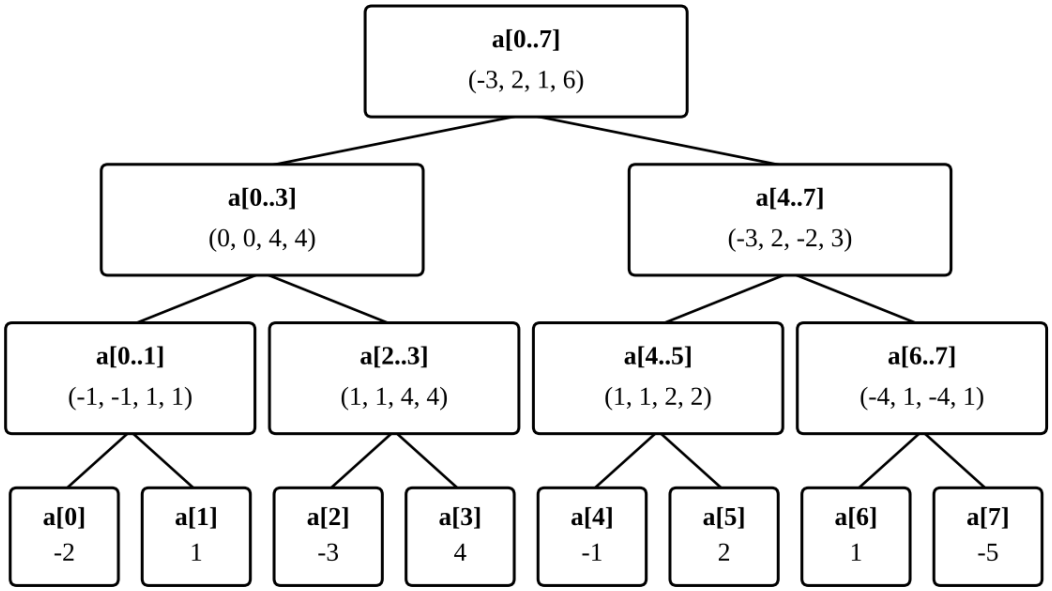
Why the combine is correct. Every non-empty block of the combined range lies wholly in the left half, wholly in the right half, or crosses the midpoint. A crossing block is a suffix of the left half followed by a prefix of the right half, so its best value is L.suffix + R.prefix. A prefix of the combined range either ends inside the left half (L.prefix) or covers the whole left half and continues into the right half (L.total + R.prefix). The suffix is symmetric.

Complete algorithm.

```
function MaxSub(a, lo, hi):
    if lo = hi:
        return (a[lo], a[lo], a[lo], a[lo])        # (total, prefix, suffix, best)
    mid = (lo + hi) // 2
    (lt, lp, ls, lb) = MaxSub(a, lo, mid)
    (rt, rp, rs, rb) = MaxSub(a, mid + 1, hi)
    total  = lt + rt
    prefix = max(lp, lt + rp)
    suffix = max(rs, rt + ls)
    best   = max(lb, rb, ls + rp)
    return (total, prefix, suffix, best)

answer = MaxSub(a, 0, n - 1).best
```

Figure 8.1 traces the algorithm on [−2, 1, −3, 4, −1, 2, 1, −5]. At the root, best = max(4, 3, 4 + 2) = 6. The crossing term uses the suffix 4 of the left half (the element a[3]) and the prefix 2 of the right half (−1 + 2 + 1), so the optimal block is a[3..6] with sum 6.



Each node shows (total, best prefix, best suffix, best subarray). Leaves return (x, x, x, x).

Figure 8.1: Trace of the joint design for n = 8

8.4 Stage 4: Validation

Student A: mathematical validation.

Recurrence. Let T(n) be the number of comparisons. A problem of size n makes two recursive calls on halves and then performs one combine, which uses 4 comparisons (one for the prefix, one for the suffix and two for the three-way maximum) and 4 additions. A single element needs no comparison.

$$\begin{aligned} T(n) &= 2\,T(n/2) + 4 \quad (n > 1), \\ T(1) &= 0 \end{aligned}$$

- **2T(n/2):** the two half-size calls. They are independent and non-overlapping, so no element is processed in more than one call at the same level.
- **+ 4:** the combine step. It touches only the four values returned by each half, so its cost does not depend on n.
- **T(1) = 0:** the base case returns without comparing anything.

Solution by unrolling ($n = 2^k$). Level i of the recursion tree has 2^i nodes, each doing a combine of cost 4, down to the level above the leaves:

$$T(n) = 4\left(2^{k-1} + 2^{k-2} + \cdots + 2 + 1\right) = 4\left(2^k - 1\right) = 4(n - 1)$$

Solution by the Master Theorem. Here a = 2, b = 2 and f(n) = 4 = Θ(1). Since $n^{\log_2 2} = n$ and $f(n) = O(n^{1-\varepsilon})$ with ε = 1, Case 1 applies and T(n) = Θ(n).

Exact count for every n. A binary tree with n leaves has n − 1 internal nodes, and each internal node costs 4 comparisons whatever the split, so T(n) = 4(n − 1) for all n, not only for powers of two. The space used is the recursion stack of depth $\lceil \log_2 n \rceil + 1$, each frame holding O(1) values, so the extra space is O(log n).

Comparison with the other designs. Student B's proposal has the recurrence T(n) = 2T(n/2) + n + 2, which is Case 2 of the Master Theorem and gives Θ(n log n); for $n = 2^k$ it equals $n \log_2 n + 2(n - 1)$ exactly.

Table 8.2: Predicted comparison counts

Approach	Recurrence or structure	Comparisons	At n = 1,024
Brute force (all blocks)	Double loop	$n(n + 1)/2 = \Theta(n^2)$	524,800
Student A's proposal	$T(n) = 2T(n/2) + 1$	$n - 1$ (result incorrect)	1,023
Student B's proposal	$T(n) = 2T(n/2) + n + 2$	$n \log_2 n + 2(n - 1) = \Theta(n \log n)$	12,286
Joint final design	$T(n) = 2T(n/2) + 4$	$4(n - 1) = \Theta(n)$	4,092
Linear scan (reference)	Single loop	$2(n - 1) = \Theta(n)$	2,046

Student B: experimental validation.

Correctness test cases. Ten fixed cases cover the single element, a single negative element, two elements, all positive, all negative, all zeros, a mixed array, an array whose optimum crosses the midpoint, an alternating array and an odd length. In addition, 3,000 random arrays (n = 1 to 60, values from −50 to 50, seed 2401) were checked against brute force.

Table 8.3: Fixed correctness cases

Test case	n	Final design	Brute force	Student A's proposal	Student B's proposal
Single element	1	7	7	7	7
Single negative element	1	−7	−7	−7	−7
Two elements	2	2	2	2	2
All positive	8	31	31	9 (wrong)	31
All negative	6	−2	−2	−2	−2
All zeros	9	0	0	0	0
Classic mixed array	9	6	6	4 (wrong)	6
Optimum crosses the midpoint	8	14	14	5 (wrong)	14
Alternating signs	7	8	8	5 (wrong)	8
Odd length	7	18	18	10 (wrong)	18

The final design and Student B's proposal matched brute force on all ten fixed cases and on all 3,000 random arrays. Student A's proposal was wrong on 2,756 of the 3,000 random arrays, which confirms that the missing crossing case is the cause of the error.

Instrumentation and datasets. A counter object is passed through the recursion and records every comparison of array-derived values and every addition. Datasets are random integers from −1,000 to 1,000 (seed 2401) with n from 8 to 1,000,000. Brute-force counts were instrumented up to n = 5,000 and are given by the exact formula $n(n + 1)/2$ above that size,

because the count is the same for every input. Timings are the median of 15, 5, 3 or 1 repetitions depending on n.

Table 8.4: Comparisons (observed; the brute-force column uses the exact formula above n = 5,000)

n	Brute force	Student B's proposal	Final design (observed)	Predicted 4(n − 1)	Linear scan
8	36	38	28	28	14
64	2,080	510	252	252	126
100	5,050	870	396	396	198
1,000	500,500	11,974	3,996	3,996	1,998
1,024	524,800	12,286	4,092	4,092	2,046
10,000	50,005,000	153,614	39,996	39,996	19,998
100,000	5,000,050,000	1,868,926	399,996	399,996	199,998
1,000,000	500,000,500,000	21,951,422	3,999,996	3,999,996	1,999,998

Table 8.5: Execution time in milliseconds (median of repeated runs)

n	Brute force (ms)	Student B's proposal (ms)	Final design (ms)	Linear scan (ms)	Final design / linear scan
16	0.011	0.013	0.011	0.005	2.2
64	0.191	0.066	0.043	0.018	2.4
256	3.044	0.340	0.183	0.074	2.5
1,000	49.34	1.685	0.797	0.312	2.6
5,000	1,559.5	9.649	3.916	1.610	2.4
10,000	5,828.3	25.13	8.87	5.402	1.6
100,000	not run	274.1	81.42	34.15	2.4
1,000,000	not run	not run	891.7	375.5	2.4

Findings.

1. **Observed equals predicted.** The final design used exactly 4(n − 1) comparisons in every row, including sizes that are not powers of two (n = 100, 1,000, 10,000). There is no discrepancy to explain, unlike Q1, because the count depends only on the number of internal nodes of the recursion tree, which is n − 1 for any split.
2. **The count does not depend on the data.** For n = 4,096 the counts were 16,380 = 4 × 4,095 for random, all-positive, all-negative, ascending and all-zero inputs.
3. **Large reduction against brute force.** At n = 10,000 brute force needs 50,005,000 comparisons and the design needs 39,996, which is 1,250 times fewer. Multiplying n

by 10 multiplies the brute-force count by about 100 and the design's count by 10. Brute force took 5,828 ms at n = 10,000 against 8.9 ms. At 0.117 µs per step, brute force at n = 1,000,000 would need roughly 16 hours (extrapolated, not run), while the design finished in 0.89 s.

4. **Reduction against Student B's proposal.** The ratio grows with log n: 3.0 times more comparisons at n = 1,024 and 5.5 times at n = 1,000,000. This is the cost of the rescan that the four-value record removes.
5. **No gain against the linear scan.** The reference scan uses exactly half the comparisons (2(n − 1)), and the design is between 1.6 and 2.6 times slower in Python (2.4 times at the two largest sizes; the scan's timing at n = 10,000 was noisy). The design matches the scan in growth rate, Θ(n), but not in the constant.

8.5 Stage 5: Cross-defence

Student A explains: what experimental evidence supports the group's design?

Three observations support it. First, correctness: the design matched brute force on all ten fixed cases and on 3,000 random arrays, while the proposal without the crossing case failed on 2,756 of the same arrays, so the tests are capable of detecting a wrong design. Second, the predicted count 4(n − 1) was observed exactly for every size from 8 to 1,000,000, which confirms the combine step is constant-time. Third, the growth class changed: multiplying n by 10 multiplies the design's count and time by about 10, while brute force grows by about 100 (5,828 ms at n = 10,000 against 8.9 ms).

Student B explains: how does the recurrence arise from the group's decomposition?

The decomposition cuts a range into two non-overlapping halves of size n/2, and each half is solved by the same procedure, which gives the term 2T(n/2). The combine step reads only the four values returned by each half and performs a fixed number of operations (4 comparisons, 4 additions), which gives the constant term. The base case returns the element itself without any comparison, so T(1) = 0. If the combine step instead rescanned the range, as in the first proposal, the constant would become a term proportional to n, and the recurrence would become T(n) = 2T(n/2) + n + 2 with the solution Θ(n log n).

8.6 Stage 6: Joint evaluation

Why the problem is suitable for Divide and Conquer. The array can be cut into two independent halves of the same problem, the answer for the whole is always one of three cases

(left, right, crossing), and the crossing case can be expressed in terms of summaries of the halves. These properties let the combine step use only the returned records.

What computational work the decomposition reduces. Brute force sums each of the $n(n + 1)/2$ blocks separately and so recomputes the same partial sums many times. After decomposition, a crossing block is obtained with one addition of a stored suffix and a stored prefix, and each half's total is computed once and reused. This reduced the work from $n(n + 1)/2$ to $4(n - 1)$ comparisons, 1,250 times fewer at $n = 10{,}000$.

Additional costs introduced by recursion. The algorithm makes $2n - 1$ calls, allocates a four-value record in each call and keeps a stack of depth $\lceil \log_2 n \rceil + 1$. In the measurements this overhead made the design between 1.6 and 2.6 times slower than the plain loop (2.4 times at $n = 100{,}000$ and $n = 1{,}000{,}000$), and it uses twice as many comparisons.

When Divide and Conquer is not appropriate. For this particular problem a single-pass scan (Kadane's algorithm) also runs in $\Theta(n)$ time with $O(1)$ space, so the design gives no asymptotic gain over the best sequential method. In general, Divide and Conquer is a poor choice when the subproblems overlap (dynamic programming reuses their results instead of recomputing them), when the combine step costs as much as the original problem (as in Student B's rescan), when a cheap summary of a subproblem does not exist, when n is so small that call overhead dominates (a cutoff to a simple loop is then needed), or when stack depth must be avoided. It remains the better choice when the two halves must be processed in parallel or in separate places, and the four-value record is exactly the node of a segment tree, which supports range queries and point updates in $O(\log n)$ time, something the linear scan cannot do.

Conclusion: what should an algorithm designer look for before choosing Divide and Conquer?

1. The problem can be split into smaller instances of the same problem that are independent and do not overlap.
2. The split can be made balanced and cheaply, so that the recursion depth is about $\log n$.
3. A solution of the whole can be assembled from solutions of the parts, and the combine step is cheaper than solving the whole problem again (ideally constant or linear).
4. The right information to return from each subproblem has been identified. If the combine step needs to look back at the data, the subproblems are probably returning too little.

5. The base case is trivial, and a cutoff to a simple method for small inputs is available to remove recursion overhead.
6. No simpler sequential or dynamic-programming method already achieves the same bound with a smaller constant, unless parallelism or a reusable structure (such as a segment tree) is a goal.

9. Appendix: Source Files and Reproduction

All programs are written in Python 3.12 and use the fixed random seed 2401. The file common.py holds the shared helpers, and each question has its own script, q1.py to q7.py. The script q7.py is self-contained. Each script prints the counts and timings reported for its question.

File	Question	Contents
common.py	All	Shared helpers used by the question scripts (seeded generator, counters, timing)
q1.py	Q1	Divide-and-conquer maximum and minimum with comparison counter; recursive binary search with probe counter
q2.py	Q2	Merge Sort and Quick Sort (last, first, median-of-three and random pivots); balanced and unbalanced input builders; partition and depth recording
q3.py	Q3	Standard digit multiplication and Karatsuba multiplication with cutoffs; multiplication, addition and call counters; crossover timing
q4.py	Q4	Conventional and Strassen matrix multiplication (pure Python and numpy); padding; operation counts and timings; memory measurement
q5.py	Q5	Brute-force and divide-and-conquer closest pair; distance counters; uniform, clustered and vertical-line datasets
q6.py	Q6	Median of Medians, QuickSelect (first and random pivot); Meet-in-the-Middle subset sum and brute-force enumeration
q7.py	Q7	Maximum subarray: brute force, linear scan, Student A's and Student B's proposals and the joint design; correctness tests; counts and timings

To reproduce a table, run the corresponding script, for example python3 q7.py. Operation counts are identical on every run, and timings vary slightly with the machine.

10. Contribution Record

Table 10.1: Contribution record

Question	Student A: primary contribution	Student B: primary contribution	Joint work
Q1	Max/Min decomposition, recurrence and mathematical comparison with $2(n-1)$; Binary Search experimental investigation	Max/Min experimental validation and discrepancy analysis; Binary Search elimination logic and termination argument	Divide-and-conquer strategy, base cases, implementation and testing of both algorithms
Q2	Quick Sort investigation: balanced and unbalanced inputs, partition sizes, recursion depth, effect of pivot choice	Merge Sort analysis, comparison of recursion structures, partition quality and tree height, path to $O(n^2)$	Implementation of both sorts, pivot strategy, experiments, joint design question
Q3	Decomposition of integers, three subproblems, recurrence and $\Theta(n^{1.585})$ solution	Crossover experiments, multiplication counts, overhead analysis	Standard and Karatsuba implementations, joint conclusion
Q4	Experimental and engineering analysis: sizes, counts, timings, crossover, overheads, memory and cache effects	Algebraic design: block decomposition, seven products, correctness, padding	Implementations, correctness verification, engineering decision
Q5	Decomposition design: ordering, division, returned values, strip step; explanation of experimental evidence	Comparative datasets, distance counts, growth and crossover; explanation of the strip argument	Review of brute force, D&C design, implementation, equivalent datasets
Q6	Median of Medians investigation; verification answer on Meet-in-the-Middle storage	Meet-in-the-Middle investigation; verification answer on pivot quality	Reference tests and cross-verification of both investigations
Q7	Independent proposal; recurrence and asymptotic validation; defence of the experimental evidence	Independent proposal; correctness tests, instrumentation and baselines; defence of the recurrence	Design comparison, final joint design, Stage 6 evaluation

Primary contribution does not imply exclusive responsibility. Both students are accountable for the complete assignment.